

Detección de Incendios Forestales con Imágenes Satelitales mediante un Pipeline de Cómputo Paralelo y Distribuido

Entrega 1: Fundamentos y Diseño del Proyecto

Allan Arnulfo Montes Monge, Esteban Gutiérrez Saborío, Maria Moroney Sole,

Marlon Mora Delgado y Robson Sthiffen Calvo Ortega

Curso TTCT0017 – Computación Paralela y Distribuida

LEAD University

Profesor: Johansell Villalobos Cubillo

San José, Costa Rica

Abstract—Los incendios forestales constituyen una amenaza creciente para los ecosistemas, la infraestructura y la vida humana, agravada por el cambio climático. La detección temprana y el análisis de grandes volúmenes de datos de teledetección requieren capacidades de cómputo que superan a los enfoques secuenciales tradicionales. Este documento corresponde a la primera entrega del proyecto del curso de Computación Paralela y Distribuida y presenta los fundamentos y el diseño de un sistema para la detección de incendios forestales a partir de dos fuentes de datos masivas: los registros tabulares de puntos de calor de NASA FIRMS (superiores a 2 GB) y un conjunto de imágenes satelitales clasificadas de aproximadamente 6.5 GB. Se justifica la selección de un ecosistema de herramientas de cómputo paralelo y acelerado por GPU (Dask, Polars, cuDF, PyTorch, RAPIDS cuML, Plotly y Dash), se definen los objetivos del proyecto, se describe el diseño del pipeline computacional por etapas y se establece el plan de trabajo para las cuatro entregas. El propósito es disponer de una base sólida que guíe la implementación posterior y el análisis de rendimiento (*speedup*, eficiencia, escalabilidad, *throughput* y utilización de recursos).

Index Terms—incendios forestales, teledetección, cómputo paralelo, cómputo distribuido, GPU, Dask, RAPIDS, aprendizaje profundo, NASA FIRMS, escalabilidad.

I. INTRODUCCIÓN

Los incendios forestales se han convertido en uno de los desastres naturales de mayor impacto a escala global. Su frecuencia, extensión y severidad han aumentado de manera sostenida durante las últimas décadas, impulsadas por el incremento de las temperaturas medias, los periodos prolongados de sequía y la acumulación de combustible vegetal. Las consecuencias abarcan la pérdida de biodiversidad, la emisión masiva de gases de efecto invernadero, el deterioro de la calidad del aire, los daños a infraestructura y, en los casos más graves, la pérdida de vidas humanas. En este contexto, la capacidad de detectar focos de calor de forma temprana y de modelar el riesgo de incendio adquiere un valor estratégico para la gestión de emergencias y la protección ambiental.

La teledetección satelital ofrece una fuente de información privilegiada para este problema, ya que permite observar de

manera continua y a escala planetaria las anomalías térmicas y las condiciones de la superficie terrestre. Sin embargo, el volumen de datos generado por las misiones satelitales modernas plantea un desafío computacional considerable. Los productos de fuego activo distribuidos por sistemas como el *Fire Information for Resource Management System* (FIRMS) de la NASA acumulan millones de registros, mientras que los repositorios de imágenes satelitales etiquetadas alcanzan varios gigabytes. Procesar, analizar y modelar estos datos con herramientas secuenciales tradicionales resulta lento e impráctico, lo que motiva el uso de técnicas de cómputo paralelo y distribuido.

La motivación de este proyecto es doble. Por un lado, abordar un problema de relevancia ambiental y social mediante el análisis de datos reales de teledetección. Por otro, aplicar de manera concreta los conceptos del curso de Computación Paralela y Distribuida: paralelismo multinúcleo, procesamiento distribuido, aceleración por GPU y aprendizaje profundo, evaluando rigurosamente el rendimiento del sistema mediante métricas como el *speedup*, la eficiencia, la escalabilidad fuerte y débil, el *throughput* y la utilización de recursos.

El presente documento constituye la primera de cuatro entregas y tiene un carácter puramente documental. Su objetivo es establecer los fundamentos teóricos y el diseño del proyecto. La Sección II revisa el trabajo relacionado; la Sección III presenta el marco teórico; la Sección IV describe los conjuntos de datos; la Sección VI justifica la selección de herramientas; la Sección VII enuncia los objetivos; la Sección VIII detalla el diseño del pipeline computacional; y la Sección XI define el plan de trabajo y el cronograma de las cuatro entregas.

II. TRABAJO RELACIONADO

La detección y predicción de incendios forestales mediante teledetección y aprendizaje automático constituye un campo de investigación activo. Los algoritmos de detección de fuego activo operacionales se basan en la firma térmica de los incendios en las bandas del infrarrojo medio. El algoritmo de Colección 6 para el sensor MODIS, propuesto por Giglio

et al. [1], es la base de los productos de fuego activo que distribuye la NASA y mejora la detección de incendios pequeños y la reducción de falsas alarmas respecto a versiones anteriores. De forma complementaria, Schroeder *et al.* [2] desarrollaron el producto de fuego activo de 375 m del sensor VIIRS, que ofrece mayor resolución espacial y mejor desempeño nocturno, permitiendo detectar focos de calor de menor tamaño.

En paralelo, el uso de aprendizaje automático y profundo para la ciencia de incendios ha crecido de manera notable. Jain *et al.* [5] ofrecen una revisión exhaustiva de las aplicaciones de aprendizaje automático en la gestión y la ciencia de incendios, abarcando la detección, la predicción de la propagación y la evaluación del riesgo. En el ámbito de la clasificación de imágenes satelitales, las redes neuronales convolucionales (CNN) se han consolidado como el enfoque dominante. Arquitecturas como las redes residuales profundas de He *et al.* [6] y la red U-Net de Ronneberger *et al.* [7] —originalmente concebida para segmentación biomédica— se han adaptado con éxito a tareas de clasificación y segmentación de cobertura terrestre y zonas quemadas.

Desde la perspectiva del cómputo de altas prestaciones aplicado a datos masivos, Dask [8] se ha establecido como un marco de referencia para la paralelización de cargas de trabajo analíticas en Python, tanto en entornos multinúcleo como distribuidos. El ecosistema RAPIDS [9] extiende este modelo a las unidades de procesamiento gráfico mediante bibliotecas como cuDF y cuML, mientras que Polars [10] ofrece un motor de *dataframes* columnar de alto rendimiento construido sobre Apache Arrow. La combinación de estas herramientas con marcos de aprendizaje profundo como PyTorch [11] habilita pipelines de extremo a extremo capaces de procesar volúmenes de datos del orden de los gigabytes con aceleración por GPU. Nuestro proyecto se ubica en la intersección de estas líneas: aplica modelos de aprendizaje automático y profundo a datos reales de teledetección de incendios, pero pone un énfasis particular en el diseño paralelo y distribuido del pipeline y en la evaluación cuantitativa de su rendimiento.

III. MARCO TEÓRICO

A. Teledetección de incendios forestales

La detección satelital de incendios se fundamenta en que los focos de calor emiten radiación con una intensidad característica en la banda del infrarrojo medio (en torno a los 4 μm). Los sensores como MODIS y VIIRS miden la temperatura de brillo de la superficie y aplican algoritmos de umbral contextual para distinguir píxeles con anomalías térmicas de su entorno. Cada detección se georreferencia y se acompaña de atributos como la potencia radiativa del fuego (FRP), la confianza de la detección, la fecha y la hora de adquisición. El producto resultante es un conjunto de datos tabular de puntos de calor que puede analizarse espacial y temporalmente.

B. Cómputo paralelo y distribuido

El cómputo paralelo busca reducir el tiempo de ejecución de un problema dividiéndolo en sub tareas que se ejecutan simultáneamente sobre múltiples unidades de procesamiento. Se distinguen, entre otros, el paralelismo multinúcleo (varios núcleos de una misma CPU con memoria compartida), el procesamiento distribuido (varios nodos que cooperan mediante paso de mensajes) y la aceleración por GPU (miles de hilos ligeros operando sobre un modelo de ejecución SIMT). El cómputo distribuido, además, permite escalar el almacenamiento y el procesamiento más allá de los límites de una sola máquina, lo que resulta indispensable cuando los datos superan la memoria disponible.

C. Métricas de rendimiento

La evaluación de un sistema paralelo se apoya en varias métricas fundamentales. El *speedup* se define como

$$S(p) = \frac{T_1}{T_p}, \quad (1)$$

donde T_1 es el tiempo de ejecución secuencial y T_p el tiempo con p unidades de procesamiento. La eficiencia se expresa como

$$E(p) = \frac{S(p)}{p} = \frac{T_1}{p T_p}. \quad (2)$$

El límite teórico del *speedup* está acotado por la ley de Amdahl [13], que para una fracción paralelizable f establece

$$S(p) = \frac{1}{(1 - f) + \frac{f}{p}}. \quad (3)$$

La ley de Gustafson [14] ofrece una perspectiva complementaria orientada al escalado del tamaño del problema, relevante para la *escalabilidad débil*. Junto a estas medidas, el análisis considerará la *escalabilidad fuerte* (tamaño de problema fijo, número de recursos creciente), el *throughput* (registros o imágenes procesadas por unidad de tiempo), el uso de memoria, la utilización de la GPU y el balance de carga entre trabajadores.

D. Aprendizaje automático y profundo

El modelado predictivo se abordará con dos familias de técnicas. Para los datos tabulares de FIRMS se emplearán algoritmos clásicos de aprendizaje automático (por ejemplo, clasificación y agrupamiento), acelerados por GPU mediante RAPIDS cuML. Para las imágenes satelitales se utilizarán redes neuronales convolucionales implementadas en PyTorch, que aprenden de forma jerárquica representaciones espaciales discriminativas para distinguir zonas con riesgo de incendio de zonas sin riesgo. El entrenamiento de estos modelos es intensivo en cómputo y se beneficia directamente de la paralelización en GPU y, potencialmente, del entrenamiento distribuido sobre múltiples dispositivos.

IV. DESCRIPCIÓN DEL DATASET

El proyecto se sustenta en dos conjuntos de datos reales, complementarios y de gran tamaño, que en conjunto superan ampliamente el requisito mínimo de 1 GB establecido por el curso.

A. NASA FIRMS: datos tabulares de puntos de calor

El *Fire Information for Resource Management System* (FIRMS) de la NASA distribuye datos de fuego activo en tiempo casi real, generalmente disponibles dentro de las tres horas posteriores a la observación satelital [3]. Los datos provienen de los sensores MODIS (a bordo de los satélites Terra y Aqua, con resolución espacial de 1 km) y VIIRS (a bordo de Suomi NPP y NOAA-20, con resolución de 375 m). Cada registro representa el centro de un píxel en el que se detectó una anomalía térmica e incluye atributos como latitud, longitud, temperatura de brillo, potencia radiativa del fuego, nivel de confianza, fecha, hora y satélite de origen.

Para este proyecto se utilizarán los archivos históricos y de tiempo casi real en formato CSV/Parquet, cuyo volumen global supera los 2 GB. La naturaleza tabular, masiva y de alta cardinalidad de estos datos los hace idóneos para ilustrar el procesamiento paralelo y distribuido de *dataframes* con Dask, Polars y cuDF, así como para el análisis exploratorio espacio-temporal de la actividad de incendios.

B. Kaggle Wildfire Prediction Dataset: imágenes satelitales

El segundo conjunto de datos es el *Wildfire Prediction Dataset* [4], disponible públicamente en Kaggle. Está compuesto por imágenes satelitales de 350×350 píxeles extraídas mediante la API de MapBox a partir de coordenadas geográficas de zonas que experimentaron incendios en Canadá, con datos de origen del gobierno de Quebec publicados bajo licencia Creative Commons Attribution 4.0. El conjunto está etiquetado en dos clases: *wildfire* (22 710 imágenes) y *no wildfire* (20 140 imágenes), para un total de 42 850 imágenes y un tamaño aproximado de 6.5 GB.

Este conjunto es apropiado para tareas de visión por computadora y aprendizaje profundo, y su tamaño justifica el uso de carga de datos paralela, aumento de datos y entrenamiento acelerado por GPU.

C. Justificación de la escala

La combinación de ambos conjuntos —más de 2 GB de datos tabulares y aproximadamente 6.5 GB de imágenes— totaliza cerca de 8.5 GB de datos heterogéneos. Esta escala es deliberada: por un lado, excede con holgura el umbral de 1 GB exigido y, por otro, plantea retos genuinos de gestión de memoria, particionado y paralelización que no surgirían con conjuntos pequeños. La heterogeneidad (datos tabulares y de imagen) permite, además, ejercitar de forma natural distintas modalidades de paralelismo: procesamiento distribuido de *dataframes* para FIRMS y aprendizaje profundo acelerado por GPU para las imágenes. La Tabla I resume las características de ambas fuentes.

TABLE I
RESUMEN DE LOS CONJUNTOS DE DATOS DEL PROYECTO

Característica	NASA FIRMS	Kaggle Wildfire
Tipo de dato	Tabular (puntos de calor)	Imágenes satelitales
Formato	CSV / Parquet	PNG (350×350)
Tamaño	> 2 GB	≈ 6.5 GB
Volumen	Millones de registros	42 850 imágenes
Etiquetas	FRP, confianza, fecha	wildfire / no wildfire
Fuente	MODIS / VIIRS (NASA)	MapBox / Quebec
Uso previsto	ML tabular + EDA	Aprendizaje profundo

V. DEFINICIÓN DEL PROBLEMA, VARIABLES E HIPÓTESIS

A. Planteamiento del problema

Los sistemas operacionales de detección satelital de incendios, como FIRMS, generan un gran número de detecciones de puntos de calor, pero no todas corresponden a incendios forestales reales: fuentes de calor intensas, reflejos, superficies calientes o ruido del sensor pueden producir *falsas alarmas* (falsos positivos). Estas falsas alarmas tienen un costo operativo real, pues desvían recursos de respuesta hacia eventos inexistentes. Reducir su proporción ha sido, de hecho, uno de los principales motores de la evolución de los algoritmos de detección [1].

El problema que aborda este proyecto es, por tanto: *a partir de la combinación de dos o más variables físicas y contextuales de cada detección, discriminar las detecciones confiables de aquellas que probablemente corresponden a falsas alarmas, con el fin de reducir la tasa de falsos positivos.*

B. Definición operativa de falsa alarma

El conjunto de datos de FIRMS contiene únicamente detecciones (no incluye una etiqueta explícita de “verdadero” o “falso”). Por ello, en este trabajo se adopta como *proxy* el atributo *confidence*, que expresa el nivel de confianza de la detección. Se define operativamente:

- **Detección válida:** registros con *confidence* media o alta.
- **Probable falsa alarma:** registros con *confidence* baja.

De este modo, el problema se formaliza como una tarea de **clasificación binaria** cuyo objetivo es identificar las probables falsas alarmas a partir de las restantes variables, sin emplear directamente *confidence* como predictora.

C. Variables del modelo

La Tabla II resume las variables. La variable *respuesta* (*target*) es la clase binaria derivada de *confidence*. Las variables *predictoras* son de dos tipos: físicas (asociadas a la firma térmica y la geometría del píxel) y contextuales (espacio-temporales). Se destaca la variable derivada *brightness* — *bright_t31*: dado que un incendio real presenta una

TABLE II
VARIABLES PARA EL MODELO DE REDUCCIÓN DE FALSAS ALARMAS

Variable	Rol	Descripción
Clase (de confidence)	Respuesta	Válida vs. probable falsa alarma
brightness	Predictora física	Temperatura de brillo (K)
bright_t31	Predictora física	Temperatura de fondo, banda 31
ΔT (brightness-bright_t31)	Predictora derivada	Contraste térmico foco-entorno
frp	Predictora física	Potencia radiativa del fuego (MW)
scan, track	Predictoras físicas	Geometría/tamaño del píxel
daynight, acq_time	Contextuales	Momento de la detección
latitude, longitude	Contextuales	Ubicación del foco

temperatura de brillo muy superior a la de su entorno, una diferencia pequeña es indicio de una posible falsa alarma.

D. Hipótesis y evaluación

Se plantea la siguiente hipótesis: *la combinación del contraste térmico ($\Delta T = \text{brightness} - \text{bright_t31}$) con la potencia radiativa (frp) permite reducir de forma significativa la proporción de falsas alarmas respecto a un criterio de umbral simple sobre una sola variable*. Para contrastarla se compararán dos enfoques: una regla de umbral univariada (línea base) y un clasificador supervisado (RAPIDS cuML). Dado que el objetivo es reducir falsos positivos, las métricas centrales son la **precisión** y la **tasa de falsas alarmas**, complementadas con el *recall* —para no descartar incendios reales— y la matriz de confusión.

VI. JUSTIFICACIÓN TÉCNICA DE LAS HERRAMIENTAS PARALELAS

La selección de herramientas responde a la necesidad de procesar de manera eficiente datos heterogéneos y de gran volumen, cubriendo las distintas etapas del pipeline con tecnologías especializadas.

Dask [8] se elige como motor de paralelismo y cómputo distribuido de propósito general. Permite particionar los *dataframes* de FIRMS en bloques que se procesan en paralelo sobre múltiples núcleos o nodos, gestionando automáticamente el grafo de tareas y la ejecución *out-of-core* cuando los datos exceden la memoria. Es la pieza central para demostrar paralelismo multinúcleo y procesamiento distribuido.

Polars [10] aporta un motor de *dataframes* columnar escrito en Rust, con ejecución multinúcleo y evaluación perezosa. Su modelo basado en Apache Arrow ofrece un rendimiento muy superior al de las bibliotecas tradicionales para operaciones de filtrado, agregación y transformación, lo que lo hace idóneo para el preprocesamiento eficiente de los datos tabulares.

cuDF (parte de RAPIDS) [9] traslada las operaciones de *dataframe* a la GPU, ofreciendo aceleraciones de uno o dos

órdenes de magnitud sobre operaciones columnares respecto a la CPU. Junto con **RAPIDS cuML**, que provee algoritmos de aprendizaje automático acelerados por GPU con una interfaz compatible con scikit-learn, permite construir un flujo tabular completo sobre la GPU.

PyTorch [11] es el marco seleccionado para el aprendizaje profundo sobre las imágenes satelitales. Ofrece diferenciación automática, ejecución acelerada por GPU y soporte para entrenamiento distribuido (*Distributed Data Parallel*), lo que habilita explícitamente el requisito de aprendizaje profundo y, potencialmente, distribuido. Para el manejo eficiente de formatos de datos en memoria se empleará **PyArrow**.

Finalmente, **Plotly** y **Dash** [12] se utilizarán para la capa de visualización interactiva, permitiendo construir un *dashboard* web que comunique los resultados del análisis y del modelado de forma dinámica. En conjunto, esta pila tecnológica cubre las seis etapas obligatorias del proyecto y satisface los requisitos de paralelismo multinúcleo, procesamiento distribuido, aceleración por GPU y aprendizaje profundo.

VII. OBJETIVOS ESPECÍFICOS DEL PROYECTO

A. Objetivo general

Diseñar e implementar un pipeline de cómputo paralelo y distribuido para la detección y el análisis de incendios forestales a partir de datos satelitales tabulares y de imagen, evaluando rigurosamente su rendimiento.

B. Objetivos específicos

- 1) **Adquisición y gestión de datos.** Adquirir, almacenar y particionar los conjuntos de NASA FIRMS (>2 GB) y Kaggle Wildfire (≈ 6.5 GB) en formatos eficientes para el procesamiento paralelo.
- 2) **Preprocesamiento paralelo/distribuido.** Implementar la limpieza, transformación y normalización de los datos tabulares con Dask, Polars y cuDF, y la carga y aumento paralelo de imágenes para el modelo de aprendizaje profundo.
- 3) **Análisis exploratorio y estadístico.** Caracterizar la distribución espacio-temporal de los focos de calor y las propiedades de las imágenes mediante análisis estadístico.
- 4) **Modelado de ML/IA.** Entrenar un modelo de aprendizaje automático sobre los datos tabulares (RAPIDS cuML) y una red neuronal convolucional sobre las imágenes (PyTorch) para clasificar zonas con y sin riesgo de incendio.
- 5) **Postprocesamiento y análisis de resultados.** Evaluar los modelos con métricas adecuadas y analizar el rendimiento computacional (speedup, eficiencia, escalabilidad fuerte y débil, throughput, uso de memoria y utilización de GPU).
- 6) **Visualización interactiva.** Desarrollar un *dashboard* con Plotly y Dash que integre los hallazgos del análisis y las predicciones del modelo.

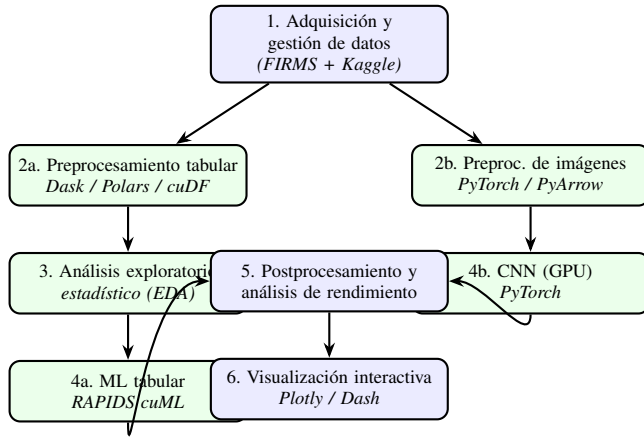


Fig. 1. Pipeline computacional del proyecto. La rama tabular (verde, izquierda) y la rama de imágenes (verde, derecha) se ejecutan en paralelo y convergen en el postprocesamiento y la visualización.

VIII. DISEÑO DEL PIPELINE COMPUTACIONAL

El sistema se organiza en seis etapas encadenadas que reflejan las etapas obligatorias del curso. La Fig. 1 muestra el flujo general, en el que la rama tabular (FIRMS) y la rama de imágenes (Kaggle) se procesan en paralelo y convergen en la capa de análisis de resultados y visualización.

Etapas 1 – Adquisición y gestión de datos. Descarga y almacenamiento de los conjuntos FIRMS y Kaggle, conversión a formatos columnar (Parquet) y particionado adecuado para el procesamiento paralelo.

Etapas 2 – Preprocesamiento paralelo/distribuido. La rama tabular limpia, filtra y transforma los registros de FIRMS con Dask, Polars y cuDF. La rama de imágenes implementa la decodificación, el redimensionamiento, la normalización y el aumento de datos con cargadores paralelos de PyTorch.

Etapas 3 – Análisis exploratorio y estadístico. Caracterización espacio-temporal de los focos de calor y análisis estadístico de las variables, apoyado en cómputo paralelo para acelerar las agregaciones.

Etapas 4 – Modelado de ML/IA. Entrenamiento de un modelo de clasificación tabular con RAPIDS cuML y de una red convolucional con PyTorch sobre GPU, con posibilidad de entrenamiento distribuido.

Etapas 5 – Postprocesamiento y análisis de resultados. Evaluación de la calidad predictiva de los modelos y, de forma central, medición del rendimiento computacional con las métricas definidas en la Sección III.

Etapas 6 – Visualización interactiva. Integración de los resultados en un *dashboard* web con Plotly y Dash, accesible y reproducible desde el repositorio del proyecto.

IX. TRABAJO PRELIMINAR: PROTOTIPO DE PREPROCESAMIENTO

Con el fin de validar la viabilidad del diseño propuesto, se desarrolló un prototipo inicial que materializa las dos primeras

TABLE III
OPERACIONES EVALUADAS EN EL PROTOTIPO DE PREPROCESAMIENTO

Operación	Métrica capturada
Carga completa	Tiempo de carga, <i>throughput</i> (filas/s y MB/s)
Filtrado por confidence	Tiempo y número de filas resultantes
Agregación por satélite	Tiempo y <i>throughput</i> de la agregación

etapas del pipeline sobre los datos tabulares de NASA FIRMS. Este trabajo se implementó en un *notebook* de Python y se encuentra versionado en el repositorio del proyecto.

A. Gestión de datos (Etapas 1)

Los registros de puntos de calor descargados de FIRMS se consolidaron en un único archivo en formato columnar Apache Parquet (`incendios_global_consolidado.parquet`). La elección de Parquet responde a su compresión eficiente, su lectura selectiva por columnas y su compatibilidad nativa con los motores de procesamiento paralelo empleados en el proyecto.

B. Preprocesamiento con Polars (Etapas 2)

La carga y el procesamiento inicial se realizaron con Polars empleando su *motor de ejecución en streaming*, que procesa los datos por bloques sin necesidad de cargar todo el conjunto en memoria de forma simultánea. El Listado 1 muestra la función de carga basada en evaluación perezosa (`scan_parquet`) y ejecución en streaming.

Listing 1. Carga en streaming del dataset consolidado con Polars.

```

1 import polars as pl
2
3 def cargar_datos_completos(ruta):
4     df = (
5         pl.scan_parquet(ruta)          # evaluación perezosa
6         .collect(engine="streaming")   # ejecución por bloques
7     )
8     return df
  
```

C. Medición de rendimiento

Sobre esta base se instrumentó una primera medición del rendimiento del preprocesamiento, capturando el tiempo de ejecución y el *throughput* tanto en filas por segundo como en megabytes por segundo. Se evaluaron tres operaciones representativas del flujo de trabajo, resumidas en la Tabla III: la carga completa del conjunto, un filtrado por nivel de confianza (*confidence*) y una agregación por satélite de origen. Los resultados se visualizaron mediante gráficos generados con Plotly.

D. Integración con el pipeline y trabajo futuro

Este prototipo confirma que la rama tabular del pipeline (Fig. 1) es viable y establece la *línea base de rendimiento* de Polars sobre la que se construirán las siguientes

entregas. En la Entrega 2 este preprocesamiento se ampliará con implementaciones equivalentes en Dask (paralelismo multinúcleo y distribuido) y cuDF (aceleración por GPU), lo que permitirá la comparación cuantitativa exigida por el análisis de rendimiento —*speedup*, eficiencia y escalabilidad— frente a una ejecución secuencial de referencia. Asimismo, se incorporará la rama de imágenes satelitales del conjunto de Kaggle, aún no abordada en este prototipo.

X. POTENCIALES CLIENTES Y APLICACIÓN COMERCIAL

Aunque este proyecto tiene un fin académico, el sistema propuesto responde a una necesidad real y podría ofrecerse, en un escenario hipotético, como un servicio de detección y análisis de riesgo de incendios forestales a diversas organizaciones. Su valor diferencial radica en la capacidad de procesar grandes volúmenes de datos satelitales de forma escalable y en tiempos reducidos, integrando detección, modelado predictivo y visualización interactiva en un mismo flujo.

Entre los potenciales clientes se identifican:

- **Organismos de gestión de emergencias y servicios forestales.** Cuerpos de bomberos forestales, protección civil y centros de comando de incendios, que se beneficiarían de la detección temprana y el mapeo rápido de focos de calor para optimizar la asignación de recursos y los tiempos de respuesta.
- **Instituciones ambientales y gubernamentales.** Ministerios de ambiente y servicios forestales nacionales (por ejemplo, SINAC/MINAE en Costa Rica o agencias provinciales en Canadá), para el monitoreo de áreas protegidas y la planificación de políticas de prevención.
- **Compañías de seguros.** Aseguradoras que requieren modelar el riesgo de incendio para el diseño de pólizas, la evaluación de siniestros y la prevención de pérdidas.
- **Sector productivo.** Empresas agrícolas y forestales, así como operadoras de servicios públicos con infraestructura en zonas boscosas (por ejemplo, tendido eléctrico), interesadas en proteger sus activos.
- **Comunidades e investigación.** Municipalidades en zonas de interfaz urbano-forestal, que requieren sistemas de alerta temprana, y centros de investigación que estudian patrones de incendios y su relación con el cambio climático.

El modelo de comercialización podría adoptar la forma de un servicio por suscripción (*SaaS*) que exponga el *dashboard* interactivo y las alertas, o de un servicio de análisis a la medida para organizaciones con necesidades específicas de cobertura geográfica o integración con sus sistemas existentes.

XI. PLAN DE TRABAJO Y CRONOGRAMA

El proyecto se estructura en cuatro entregas, con pesos del 20 %, 30 %, 40 % y 10 % respectivamente. El cronograma se ajusta al calendario del cuatrimestre en curso, que abarca de la semana 7 (fecha de esta primera entrega) a la semana 15 (cierre del curso). La Tabla IV resume las semanas, fechas

TABLE IV
PLAN DE TRABAJO Y CRONOGRAMA DE LAS CUATRO ENTREGAS

Ent.	Peso	Semana / fecha	Objetivos y entregables
1	20 %	Sem. 7 (≈27 jun)	Fundamentos y diseño. Informe IEEE con introducción, marco teórico, descripción del dataset, justificación de herramientas, objetivos, diseño del pipeline y cronograma. (Documento actual.)
2	30 %	Sem. 8–10 (≈18 jul)	Implementación inicial y validación parcial. Adquisición y gestión de datos, preprocesamiento paralelo/distribuido funcional y primeros resultados de análisis exploratorio. Mediciones preliminares de rendimiento.
3	40 %	Sem. 11–14 (≈15 ago)	Proyecto final completo. Modelos de ML y aprendizaje profundo entrenados, postprocesamiento, análisis completo de rendimiento (<i>speedup</i> , eficiencia, escalabilidad, throughput, GPU) y dashboard interactivo. Informe técnico final.
4	10 %	Sem. 15 (≈22 ago)	Presentación. Exposición de la metodología, los resultados y las conclusiones del proyecto.

aproximadas, objetivos y entregables de cada fase. El trabajo se coordinará a través de un repositorio GitLab que contendrá el código, los *notebooks*, el README, los enlaces a los datos, los resultados y el *dashboard*.

A. Distribución de responsabilidades

El equipo, integrado por Allan Arnulfo Montes Monge, Esteban Gutiérrez Saborío, María Moroney Sole, Marlon Mora Delgado y Robson Sthiffen Calvo Ortega, distribuirá las tareas de manera equilibrada a lo largo de las etapas del pipeline (gestión de datos, preprocesamiento, análisis, modelado tabular, modelado de imágenes, análisis de rendimiento y visualización), garantizando que todos los integrantes participen tanto en la implementación como en la redacción de los informes.

XII. CONCLUSIÓN

Esta primera entrega ha establecido los fundamentos y el diseño del proyecto de detección de incendios forestales con imágenes satelitales. Se ha justificado la relevancia del problema, se han descrito dos conjuntos de datos reales que en conjunto superan los 8 GB, se ha seleccionado y argumentado una pila de herramientas de cómputo paralelo y acelerado por GPU, y se ha definido un pipeline computacional de seis etapas junto con un plan de trabajo para las cuatro entregas. Este diseño proporciona una base coherente para la fase de implementación, en la que se materializarán

el preprocesamiento distribuido, el modelado y, de manera central, la evaluación rigurosa del rendimiento del sistema.

REFERENCES

- [1] L. Giglio, W. Schroeder, y C. O. Justice, “The collection 6 MODIS active fire detection algorithm and fire products,” *Remote Sensing of Environment*, vol. 178, pp. 31–41, 2016.
- [2] W. Schroeder, P. Oliva, L. Giglio, y I. A. Csizar, “The New VIIRS 375 m active fire detection data product: Algorithm description and initial assessment,” *Remote Sensing of Environment*, vol. 143, pp. 85–96, 2014.
- [3] NASA, “Fire Information for Resource Management System (FIRMS),” NASA Earthdata. [En línea]. Disponible: <https://firms.modaps.eosdis.nasa.gov/>
- [4] A. Aaba, “Wildfire Prediction Dataset (Satellite Images),” Kaggle, 2022. [En línea]. Disponible: <https://www.kaggle.com/datasets/abdelghaniaaba/wildfire-prediction-dataset>
- [5] P. Jain, S. C. P. Coogan, S. G. Subramanian, M. Crowley, S. Taylor, y M. D. Flannigan, “A review of machine learning applications in wildfire science and management,” *Environmental Reviews*, vol. 28, no. 4, pp. 478–505, 2020.
- [6] K. He, X. Zhang, S. Ren, y J. Sun, “Deep residual learning for image recognition,” en *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [7] O. Ronneberger, P. Fischer, y T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” en *Proc. Int. Conf. Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [8] M. Rocklin, “Dask: Parallel computation with blocked algorithms and task scheduling,” en *Proc. 14th Python in Science Conf. (SciPy)*, 2015, pp. 130–136.
- [9] RAPIDS Development Team, “RAPIDS: Collection of libraries for end-to-end GPU data science,” NVIDIA, 2023. [En línea]. Disponible: <https://rapids.ai/>
- [10] R. Vink y col., “Polars: Lightning-fast DataFrame library for Rust and Python,” 2023. [En línea]. Disponible: <https://pola.rs/>
- [11] A. Paszke y col., “PyTorch: An imperative style, high-performance deep learning library,” en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.
- [12] Plotly Technologies Inc., “Dash: A Python framework for building analytical web applications,” 2023. [En línea]. Disponible: <https://dash.plotly.com/>
- [13] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” en *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.
- [14] J. L. Gustafson, “Reevaluating Amdahl’s law,” *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.